

Certified but Compromised: Breaking and Fixing Lightweight Proofs of Inference

Final Report — Trustworthy Machine Learning, Spring 2026

Ofek Lavi
Tel Aviv University
Tel Aviv, Israel
ofeklavi@mail.tau.ac.il

Erel Barzilay
Tel Aviv University
Tel Aviv, Israel
erelbarzilay@mail.tau.ac.il

Edo Kadosh
Tel Aviv University
Tel Aviv, Israel
edo@mail.tau.ac.il

Abstract

When a large model runs in the cloud, its users cannot tell whether an answer came from the agreed weights. zkSNARK proofs of inference settle this but take minutes per query on large models, so Anchuri et al. (SaTML 2026) proposed a lightweight alternative in which the client re-checks one random path through a committed computation. Its guarantee covers a provider that runs some other model. We show that a provider that runs the committed model but changes a single neuron is accepted 99.8% of the time on an MNIST network. Applied only to triggered inputs, the edit is a backdoor that leaves the committed weights clean, and on the real OPT-6.7B one neuron changes the next token of every prompt we tried. Against an adversary who knows the sampler, no rule that ignores the claimed values does better than uniform sampling, and the best rule we found that uses them has a blind spot. Our defence checks every layer with one random linear combination (Freivalds’ check), in exact integer arithmetic, against a Reed–Solomon/Merkle commitment to the weights, and accepts a wrong output with probability at most $2^{-\lambda}$ without a trusted setup. It rejected all 2,895 attacks, on models from LeNet-5 to Llama-2-13B and on truncated builds of larger ones. Its prover is one to four orders of magnitude faster than published zkSNARK provers, and 157× faster than zkLLM’s own code on our GPU for Llama-2-7B. With optimisations that keep the bound, it also verifies faster and sends less, non-interactively, than zkCNN on LeNet-5 and, for GPT-2’s next-token logits, than DeepProve’s transparent configuration. Its remaining cost is a proof that grows with the activations.

Keywords

verifiable inference, proof of inference, integrity attacks, Freivalds’ algorithm, zkML

1 Introduction

Consider a company that pays a cloud provider to serve a licensed or fine-tuned language model to its users. Nothing in a response shows that the provider ran that model with the agreed weights rather than a cheaper quantised copy, a model with a few edited weights, or one that misbehaves on certain inputs. The company cannot re-run the model to find out, since it lacks the weights and the hardware; that is why it outsourced inference. A *proof of inference* closes this gap. With each answer the provider sends a proof, and the client runs a check, much cheaper than inference, that fails with high probability unless the answer was computed with the agreed weights on this input.

zkSNARKs [6, 13, 20, 24, 26] prove that the answer is correct and can keep the weights secret. Their proofs are usually small and fast to check, but producing one takes far longer than inference; zkLLM needs about ten minutes on an A100 for one Llama-2-7B prompt [26]. Anchuri et al. [2] proposed a much lighter approach. The provider commits to the full execution trace with a Merkle tree, and the client checks the computation along *one random path* from an output neuron back to the input, against a commitment to the weights. Proving drops from minutes to milliseconds. A different model produces a trace that differs from the correct one in many places, so a random path is likely to hit one of them, and Anchuri et al. accordingly prove *other-model* soundness, against a provider that honestly runs some different model. They note that one path cannot catch a single changed node with probability above $1/N$ in a layer of width N , and they leave full soundness for LLMs open.

What we did. We reimplemented the protocol (no code was released), reproduced its main claims, and asked what a provider can do if it keeps the committed model and changes only what it needs to. It can overwrite *a single neuron* and recompute everything after it honestly (Figures 1a and 1b). The trace is then inconsistent at one node, which a random path visits with probability $1/N$, so almost every tampered answer is accepted. Applied only to inputs that carry a trigger, the edit is a backdoor in the serving code, which leaves the committed weights clean and answers clean queries bit for bit as before. The attack carries over to a real 6.7B-parameter model. Auditing does not close the gap, because the provider chooses which queries to tamper with, and tampering with k of them is caught with probability at most k/N . Nor does sampling more paths, or sampling adaptively, as Anchuri et al. suggest. Uniform sampling is already optimal among rules that ignore the claimed values, the best rule we found that uses them has a blind spot, and reaching 2^{-40} forces the path protocol to open essentially the whole model and trace.

This led us to a different design (Figure 1c). The verifier tests one random linear combination of all the neurons of every layer (Freivalds’ algorithm [12]). When it does not hold the weights, it checks the combination against a Reed–Solomon-encoded, Merkle-committed copy of them in the style of Ligerio [1], so it still stores 32 bytes per weight matrix. The model runs as an int8 network whose results are bit-identical on the provider’s GPU and on the client, which makes the test exact. A tampered value anywhere in the network is then caught with probability $1 - 2^{-\lambda}$ instead of $1/N$, and the prover’s extra work grows with the model but not with the input. At equal or higher security our proofs are also the smaller ones: catching the single-neuron attack on a 64-token

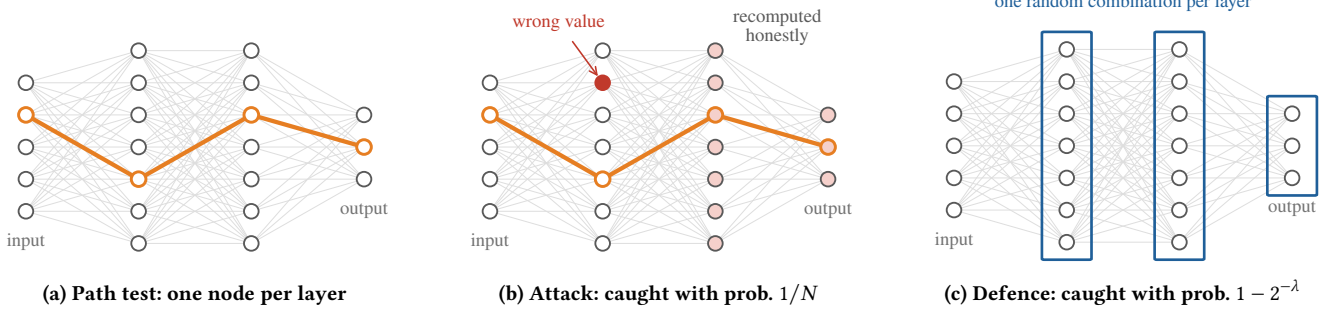


Figure 1: A single tampered neuron, with everything after it recomputed honestly, leaves one inconsistent node: a random path rarely visits it (b), while one random combination per layer catches it (c).

Llama-2-7B prompt with probability $1 - 2^{-40}$ takes paths that open about 13.7 GB, while our proof at 2^{-128} is 762 MB, or 329 MB after the optimisations of Section 4.3. Spot-checking a few neurons exactly leaves the rest of the network unconstrained; one random combination per layer constrains all of it.

Contributions.

- **An attack** that exploits the $1/N$ limit acknowledged in [2]. One tampered neuron is accepted with probability $1 - 1/N$ (99.6–99.8% on MNIST), works as a trigger backdoor that leaves the weights and clean answers unchanged, and changes the next token of 40 of 40 prompts on the real OPT-6.7B.
- **A bound on sampling-based fixes**: no value-blind sampler beats uniform sampling, the contribution-weighted sampler is blind to zeroed neurons, and 2^{-40} requires opening essentially the whole model and trace.
- **A full-soundness protocol** for whole CNNs and transformers, with error $2^{-\lambda}$ and no trusted setup, and optimisations that keep the bound and shrink the committed-weights proof 2.0–4.2× for images and prompts of up to 512 tokens.
- **A same-architecture evaluation** on 6 image classifiers and 10 LLM architectures up to 13B parameters, against [2], 17 published systems and zkLLM’s code on our GPU. On matched models our prover is 53–32,000× faster than the zkSNARK provers, and on LeNet-5 and on GPT-2’s next-token logits the optimised protocol beats zkCNN and DeepProve’s transparent configuration on all three costs (prover time, verifier time and proof size). Code and raw measurements are released.

2 Related Work

zkSNARK proofs of inference. A long line of work compiles neural networks into zero-knowledge proofs. For CNNs these include sum-check/GKR provers [15] such as zkCNN [20] and zkPyTorch [30], and pairing- or Plonk-based systems such as vCNN [18], ZEN [9], ZENO [10], ZKML [6], EZKL [34] and Bionetta [33]; Mystique [29] is an interactive, VOLE-based alternative. For LLMs they include zkLLM [26] (also interactive), zkGPT [24], DeepProve [13], ZKTorch [5] and Jolt Atlas [4]. They

give computational soundness and mostly small proofs, but their provers are far slower than inference: from half a second for LeNet-5 to minutes or hours for larger models.

Freivalds-style checks. SafetyNets [14] uses interactive proofs for networks with quadratic activations. Slalom [28] checks every linear layer of a GPU computation from inside an SGX enclave with Freivalds’ algorithm, in exact arithmetic modulo a 24-bit prime and optionally with a precomputed secret challenge; TwinShield [31] and VeriAttn [7] extend this to attention inside TEEs. Two recent works need no enclave. Maverick [21] combines sparse Freivalds checks with a linear code, for a client that knows the weights and draws a fresh challenge per query. LAMP [17] lets a verifier that holds only a commitment check one matrix product, using Freivalds’ check and Reed–Solomon proximity testing inside a Groth16 SNARK.

Sampling, fingerprints and optimistic protocols. Anchuri et al. [2] spot-check random paths and argue security through trace separation; SLP [19] samples which layers receive a SNARK. TOPLOC [23] and SVIP [27] fingerprint hidden states, with empirical guarantees. Verde [3], opML [8] and TAO [32] resolve disputes and assume an honest party or an active challenger.

Where we fit. To our knowledge, ours is the first protocol with full soundness $2^{-\lambda}$ in the setting of [2], a client holding a 32-byte commitment per weight matrix, for complete CNNs and transformers without a SNARK, a TEE or a structured reference string. Its parts are known: Freivalds’ check, exact integer arithmetic and client-side non-linear layers (Slalom, Maverick), and a Reed–Solomon-encoded Merkle commitment opened at random columns (Ligero, LAMP). We contribute the attack, the bound on sampling, their composition into a layer-chained protocol with an end-to-end soundness proof, optimisations that keep that bound, and a same-architecture comparison.

3 Breaking Path Sampling

3.1 Setting and threat model

A model M has a public architecture and secret or public weights. The provider acts as the prover and the client as the verifier. Before any query the client obtains a commitment C_M to the weights, for example from the model owner. As in [2], we assume that C_M is computed honestly; unlike a SNARK’s trusted setup it involves no

secret randomness, and anyone who holds the weights can recompute it. For each query x the provider returns a claimed output y and a proof, and may deviate from the protocol arbitrarily. We require *completeness* (honest answers are always accepted) and *full soundness*: any $y \neq M(x)$ is rejected except with probability ε . Anchuri et al. prove only *other-model soundness*, in which the adversary must honestly run some different model. We consider three trust settings. In C the client holds only C_M (32 bytes per weight matrix), as in [2] and the zkSNARKs; in K it knows the weights; and K_{pre} is K with a secret challenge precomputed once per model, as in Slalom’s preprocessing mode.

3.2 The path test and the single-neuron attack

In the protocol of [2] (RandPathTest) the prover commits to every activation a_j of the forward pass with a Merkle tree. The verifier chooses a random output neuron and walks back to the input, moving to a random parent at each layer. At every node j on the path it opens the parents’ activations and the weights w_{ij} from C_M and checks $a_j = \phi(\sum_{i \in G_j} w_{ij} a_i)$, where G_j is the set of parents of j and ϕ the activation. A path opens one weight row per layer, so the proof and the prover’s work stay small.

The single-neuron attack. Our adversary runs the correct model on x , picks one neuron j , and replaces a_j by a value that makes the output wrong (untargeted) or equal to a class of its choice (targeted). It then recomputes all later layers *with the true weights*. Every later node is consistent with its parents and every earlier node is untouched, so j is the only node whose local check fails. The verifier notices the attack only if its path visits j , which happens with probability $1/N$ in a layer of width N , or $1 - (1 - 1/N)^s$ with s paths. Applied only to inputs that contain a trigger, the attack becomes a backdoor in the serving code. The prover keeps the committed weights and answers clean queries bit for bit honestly, so no inspection of the weights reveals it and the clean-accuracy gap is zero by construction.

3.3 Why sampling cannot fix it

Anchuri et al. suggest sampling more paths, or sampling where it matters instead of uniformly. Against an adversary who knows the rule, neither helps if the rule ignores the claimed values.

THEOREM 3.1 (UNIFORM SAMPLING IS MINIMAX-OPTIMAL). *Fix a layer, let U be the set of its neurons whose change can flip the output, and consider any sampler that opens at most s nodes of the layer, chosen independently of the claimed trace. Let $\pi(v)$ be the probability that node v is opened. Then $\min_{v \in U} \pi(v) \leq s/|U|$, with equality only if π is uniform on U .*

An adversary who knows the sampler tampers with a node of U that minimises π , so it is detected with probability at most $s/|U|$. The bound holds because a minimum is at most the average and the $\pi(v)$ sum to at most s . Constant detection therefore needs $s = \Omega(|U|)$ openings per layer, linear in the width, which is the cost the protocol was designed to avoid. Samplers that look at the claimed values are not safe either. The most effective one we found steps to parent i with probability proportional to its contribution $|w_{ij} a_i|$, so

Table 1: Attacks on the path protocol (one path). The forgeries of [2] corrupt hundreds of the MLP’s 778 trace nodes and are always caught; ours leave 1–14 inconsistent nodes and are almost never caught.

Attack	inconsistent nodes	accepted
<i>Forgeries of [2], MNIST MLP</i>		
Substitute model M	≈ 574	0.000
Inverse transform	591	0.000
Logit swap	256	0.000
Gradient reconstruction	≈ 778	0.000
<i>Our attacks, MNIST MLP</i>		
Single neuron, layer 1	1	0.998
Single neuron, layer 2	1	0.996
Backdoor, triggered inputs	1	0.998
Stealthy (in natural range)	≈ 5	0.989
Zeroing vs. contribution sampler	≈ 14	1.000
<i>Our attacks, OPT-6.7B (real weights)</i>		
One FFN neuron (40/40 prompts)	1	0.99994
Backdoor, target “back” (40/40)	1	0.99994

it never visits a neuron whose claimed activation is zero, and an adversary who flips the output by zeroing a few neurons is never detected. A floor δ , sampling by $|w_{ij}|(|a_i| + \delta)$, removes this blind spot but leaves detection at a few percent (Section 3.4), and a forgery that keeps every value within its neuron’s natural range gives a range check nothing to flag.

3.4 The attack in practice

Reproduction. Our implementation behaves as Anchuri et al. describe. On MNIST versions of four of their substitute-model settings (three from their experiments, one from their motivation), honest provers were accepted in all 3,000 runs of each. Substitute models were caught in 99.8–100% of runs, including an 8-bit quantised copy that agrees with M on all 150 evaluated queries. Exact equality, by contrast, accepts an honest prover in only 21–37% of runs, because a layer-wide matrix product and a single-neuron dot product round differently, so the prescribed tolerance of 10^{-4} is necessary for completeness.

The single-neuron attack. Table 1 gives the number of inconsistent trace nodes and the exact acceptance probability, computed by dynamic programming over the paths. The forgeries of [2] (its substitute model, the inverse-transform and logit-swap forgeries of its Appendices E and F, and the gradient reconstruction of its Section 7.2) corrupt hundreds of nodes and are always caught. One tampered neuron is accepted with the predicted probability $1 - 1/N$, and in 99.96% of 2,300 end-to-end runs. Triggered by a 3×3 bright patch, the backdoor returns the honest output on every clean query and succeeds on every triggered query not already in the target class. Keeping each forged value below its neuron’s 95th percentile takes about five neurons and still passes 99% of the time, and with 250 paths the attack is still accepted 61% of the time.

Other samplers. Against an adversary who knows the rule, uniform sampling detects 0.20% of attacks, a static importance score

0.11%, and gradient saliency, which a verifier cannot compute without the model, 0.016%. Contribution weighting detects nothing: zeroing about 14 of the 512 neurons flips the prediction, and end-to-end runs accepted 2,000 of 2,000 attacks. Zero is also these neurons' most common value (65% of inputs), so activation statistics do not flag the attack. A floor $\delta = 1$ raises detection to 2% (10 $\times$ uniform); other floors give 0.9–6 $\times$.

Larger models. On every CNN and every test query, changing one penultimate neuron suffices to flip the prediction. One path catches it with probability 1/84 (LeNet-5) to 1/512, and the least-visited neuron with probability as low as 1/28 million; 2^{-40} takes 2,316–14,182 paths, which open the whole model and trace (Figure 3). On Llama-2-7B one path detects the attack with probability 1/11,008, and 2^{-40} needs about 305,000 paths (≈ 13.7 GB with the fp16 weights of [2], at 64 tokens).

A real LLM. On the real OPT-6.7B (fp16), the attacker ranks a few FFN neurons of layers 8, 16, 24 and 31 at the last position, by gradient and by their direct effect on the logits, and finds the smallest change to one of them that changes the next token. One neuron sufficed for all 40 prompts. A fixed-target backdoor is more constrained, because the final LayerNorm makes each neuron force only its own *saturation token* as its activation grows: the last layer's 16,384 neurons reach 6,187 of the 50,272 tokens. The backdoor succeeds for a reachable target ("back": 40/40 triggered prompts, clean prompts unchanged) and fails for "hacked" (0/40). A per-neuron range check would flag its values, which exceed the neurons' natural range, but the path protocol has none; one path detects the backdoor with probability 1/16,384, and 2^{-40} needs 454,248 paths.

4 Checking Every Layer

Our verifier tests one random linear combination of all the neurons of each weight layer. A wrong value anywhere in the layer changes this combination except with probability $1/p$, so the test needs no assumption about where the adversary tampers. We build it in four steps.

4.1 Construction

Step 1: an exact integer model. A random combination is a reliable test only if both sides compute exactly the same numbers, so the model runs in integers. Following integer-only inference [16], each weight layer computes $Z = WX + b$, where W is int8 with one scale per output channel, X is int8 with one scale per tensor, and BatchNorm is folded into W and b . The next layer's input is obtained by fixed-point requantisation followed by the activation or pooling, and convolutions become matrix products through im2col. On the prover's GPU the products are computed in float32 chunks of at most 1,024 terms, so every partial sum stays below 2^{24} in magnitude and is exact, and the prover's results are bit-identical to the verifier's. Exactness also removes the floating-point tolerance of the path protocol, which leaves an attacker room to hide small changes.

Step 2: only weight products need a proof. For every weight layer ℓ the prover sends the claimed pre-activations Z_ℓ , the *claims*. The verifier recomputes every operation that does not involve the weights from the claims of earlier layers: requantisation, ReLU,

pooling, residual additions, LayerNorm/RMSNorm, GELU/SiLU (by table look-up), RoPE, softmax, and the attention products $QK^\top$ and PV . Each weight layer's input X_ℓ is therefore the verifier's own computation, and what remains is to check $Z_\ell = W_\ell X_\ell + b_\ell$ for every ℓ . Because each X_ℓ is derived from the claims of earlier layers, the checks are chained, so a prover cannot pass one layer's check with one value and feed another into the next.

Step 3: Freivalds' check. Write $A_\ell = [W_\ell | b_\ell]$, a matrix with N rows (one per output neuron) and row length k , and let T be the number of columns of X_ℓ (pixels or tokens); we drop the subscript ℓ from N , k and T when the layer is clear. The verifier draws a random $\chi \in \mathbb{F}_p^N$ and checks

$$\chi^\top Z_\ell = \underbrace{(\chi^\top A_\ell)}_{u_\ell} [X_\ell; \mathbf{1}], \quad (1)$$

where we call u_ℓ the *folded row*. Let $D = Z_\ell - A_\ell [X_\ell; \mathbf{1}]$; the two sides of (1) differ by $\chi^\top D$. If $D_{im} \neq 0$, entry m of $\chi^\top D$ equals $\chi_i D_{im}$ plus terms that do not involve χ_i , so exactly one value of χ_i makes it vanish, and a wrong layer passes with probability at most $1/p$, or p^{-r} with r independent vectors. This holds however many values are wrong and wherever they are: a single tampered neuron is caught as reliably as a wholly wrong layer. Both sides cost $O((N+k)T)$, against $O(NkT)$ for recomputing the layer.

We work in the BabyBear field $\mathbb{F}_p$, $p = 15 \cdot 2^{27} + 1 \approx 2^{31}$. Integers map into $\mathbb{F}_p$ without wrap-around as long as they are small, and both sides enforce this. The verifier range-checks every claim ($|z| < 2^{29}$), and setup checks that every honest pre-activation is in the same range ($128 \sum_c |w_{ic}| + |b_i| < 2^{29}$ for every row i , since int8 inputs are at most 128 in magnitude). A wrong claim and the correct value then differ by less than $2^{30} < p$, so they remain distinct in $\mathbb{F}_p$.

Step 4: committed weights. In setting C the verifier cannot compute u_ℓ , so the prover sends it. Opening t random entries of u_ℓ against the weights would be weak: a prover who changes one of the k entries escapes with probability $1 - t/k$. We therefore commit to an *encoded* copy of the weights, in the style of Ligerio [1]. At setup every row of A_ℓ is padded with zeros to a power of two $\bar{k} \geq k$ and Reed–Solomon encoded [25] to length n_ℓ ($4\bar{k}$ in the basic protocol), giving $\hat{A}_\ell$, and C_M holds one Merkle root [22] per weight matrix, over the columns of $\hat{A}_\ell$. Encoding is linear, so $\chi^\top \hat{A}_\ell = \text{Enc}(\chi^\top A_\ell)$. The verifier encodes the claimed u_ℓ itself, opens t random columns c , and checks $\chi^\top \hat{A}_{\ell,c} = \text{Enc}(u_\ell)_c$. If u_ℓ is wrong, $\text{Enc}(u_\ell)$ and $\text{Enc}(\chi^\top A_\ell)$ are distinct codewords that agree on at most $k - 1$ of the n_ℓ positions, so t distinct columns all miss the difference with probability at most $((k - 1)/n_\ell)^t < 4^{-t}$.

4.2 Protocol and soundness

Setup happens once per model: the owner encodes and commits the weights and publishes C_M . Figure 2 shows one query. After the claims (message 2) the verifier range-checks them, recomputes every non-weight operation, which yields each X_ℓ and checks that they lead to y , and draws r vectors χ per layer (3). It checks (1) with the returned u_ℓ (4), draws t column positions (5), and checks the opened columns against C_M and $\text{Enc}(u_\ell)$ (6). Since the claims are fixed before χ is drawn and u_ℓ before the columns are chosen,

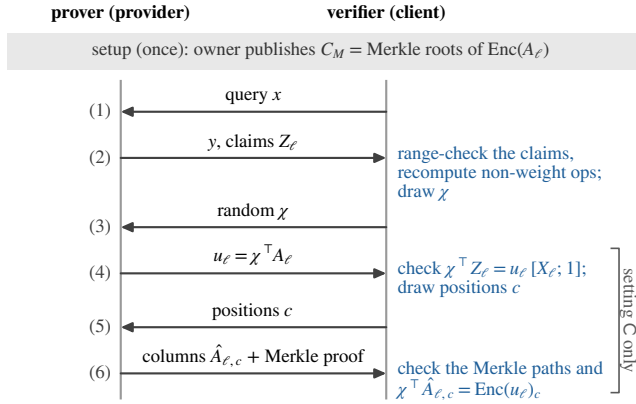


Figure 2: One query in setting C. The claims are fixed before χ is drawn and u_e before the columns are chosen; in K and K_{pre} messages (4)–(6) are omitted.

the prover cannot adapt a wrong value to the randomness that tests it.

THEOREM 4.1 (SOUNDNESS). *If C_M is honest and the prover cannot find a SHA-256 collision, then for any claimed $y \neq M(x)$ the verifier accepts with probability at most $\varepsilon = \sum_{\ell} [p^{-r} + (\frac{k_{\ell}-1}{n_{\ell}})^t]$.*

Sketch. If $y \neq M(x)$, some claim is wrong; let ℓ be the first layer with a wrong claim. Its input X_{ℓ} is correct, because the verifier computed it from correct claims. If u_{ℓ} is correct, (1) fails except with probability p^{-r} (Step 3). If u_{ℓ} is wrong, a column check fails except with probability $((k_{\ell}-1)/n_{\ell})^t$ (Step 4). A union bound over the layers finishes the proof. $\square$

Parameters. For a target λ and L weight layers we set $\beta = \lambda + \log_2(2L)$, $r = \lceil \beta / \log_2 p \rceil$ and $t = \lceil \beta / 2 \rceil$; since $n_{\ell} \geq 4k_{\ell}$, each of the $2L$ terms of ε is at most $2^{-\beta}$, so $\varepsilon \leq 2^{-\lambda}$ (for VGG-16 at $\lambda=128, r=5$ and $t=67$). Deriving the challenges from the transcript with SHAKE-256 (Fiat–Shamir [11]) makes the protocol non-interactive; 64 extra bits in β keep a prover who tries up to 2^{64} transcripts below $2^{-\lambda}$.

Known weights. In K the client computes u_{ℓ} itself and messages (4)–(6) are omitted. In K_{pre} it computes the secret χ and u_{ℓ} once per model, so the proof is the claims alone. The secret χ exists before the claims but the prover never sees it, so each query is falsely accepted with probability at most $\sum_{\ell} p^{-r}$, and any of Q queries that reuse χ with probability at most $Q \sum_{\ell} p^{-r}$.

4.3 Costs and optimisations

Costs. The prover runs the forward pass, computes u_{ℓ} and opens t columns, which it recomputes from W_{ℓ} ; its extra work, $O((r+t) \sum_{\ell} N_{\ell} k_{\ell})$, does not depend on the number of pixels or tokens. A proof consists of the claims, $4 \sum_{\ell} N_{\ell} T_{\ell}$ bytes, which grow with the input, and of the folded rows, opened columns and Merkle hashes, which do not. Unlike zkCNN’s or zkLLM’s proofs, which stay between tens and a few hundred kilobytes, ours grows with the activations.

Optimisations. The parameters above were chosen to keep the soundness argument short, and they leave the proof larger than it needs to be. We call the protocol so far the *basic* protocol, and the one with the four changes below the *optimised* protocol. The changes alter only how the weights are committed and how messages are sent, and Theorem 4.1 holds in the form

$$\varepsilon \leq \sum_j \left[p^{-r} + \prod_{i < t_j} \frac{m_j - 1 - i}{n_j - i} \right] \quad (2)$$

over the committed matrices j , with message length m_j , codeword length n_j and t_j distinct opened columns. This product bounds the chance that all t_j positions fall among the at most $m_j - 1$ where two distinct codewords agree. (i) *Columns and codes.* Each matrix opens the fewest columns that keep its term below $2^{-\beta}$, and matrices with codewords of equal length share one Merkle tree and multiproof. A public *planning rule* picks from the model’s shapes the codeword lengths that minimise the bytes per query within a one-time setup of $\max(2 \times \text{basic}, 2^{34})$ encoded entries. LeNet-5’s matrices then share one tree of length 2^{18} and open 15 columns instead of 66 in each of five trees. (ii) *Transposed matrices.* Committing the encoded rows $\hat{A}'$ of A^T lets the verifier combine positions instead of neurons: it draws $\chi' \in \mathbb{F}_p^T$, computes $z = Z\chi'$ and $w = [X; 1]\chi'$ itself, and checks $\text{Enc}(z)_c = (w^T \hat{A}')_c$ on t opened columns of k entries. No folded row is sent, and the opening shrinks from $rk + tN$ to tk field elements, which pays off for wide layers such as GPT-2’s output layer ($N=50,257, k=768$); projections of one input (query, key and value; gate and up) share one such matrix. (iii) *Decoders.* Embedding tables are committed as Merkle trees over their rows, so a wrong row needs a hash collision, and for a next-token output both parties compute the last block at the last position only. (iv) *Compact encoding.* Claims travel at 16–19 bits instead of 32 and field elements at 31; the verifier decodes before checking, and Fiat–Shamir absorbs the encoded bytes. The plan (lengths, layouts, groups) follows from public shapes and is part of C_M . Separately, *verifier engineering* evaluates $\text{Enc}(u_{\ell})$ only at the opened positions, batches its exact products, computes attention in int32 like the prover, and can stream the claims to a GPU, without changing a proof byte.

5 Results

5.1 Setup

Models. LeNet-5 (MNIST) and VGG-11, VGG-16 and ResNet-18 (CIFAR-10) are the CNN architectures that zkCNN, ZKML and Bionetta report on. Some published versions are smaller variants (ZKML’s ResNet-18 has 281K parameters), so Table 5 compares only models that match ours. A 224×224 dog-vs-cat ResNet-18 trained on ImageNet classes stands in for the unreleased model of [2], and a dog-vs-squirrel one for its substitute $\tilde{M}$; the MNIST MLP (784–512–256–10) is the network of Section 3.4. We trained all of them, and int8 quantisation changed accuracy by at most 0.4 points. For language models we use the shapes of GPT-2, OPT-125M to OPT-13B, Llama-2-7B/13B and Qwen3-4B with random int8 weights, since the costs depend only on the shapes (except the compact encoding of Section 4.3, measured on the same random

weights). Every decoder up to 13B is built in full, at up to 2,048 tokens.

Real weights. To check that the integer graph is a usable language model, we load the real OPT-125M, OPT-1.3B and OPT-6.7B checkpoints into the same graph (plus one bias on the LM head). On 4,064 tokens of WikiText-2 their perplexity is 73.8, 37.4 and 36.0, against 64.7, 34.9 and 27.1 in fp32. This needs two public constants, the gain G of the integer normalisation (4, 3 and 2; the benchmark’s $G=32$ clips OPT’s outlier features, giving 326 to 23,849) and the activation-calibration percentile. We chose both on the evaluation text, so these perplexities are slightly optimistic.

Hardware and baselines. The prover is one NVIDIA L40S (48 GB) and the verifier 8 threads of an AMD EPYC 9334; times are medians over a run’s queries. Of the 30–70B shapes only 1- and 2-block builds fit, from which we extrapolate proof sizes (to within 0.01%) but no times. A second run on an RTX 2080 Ti reproduced all 1,978 hardware-independent values. Published costs are taken as reported on their own hardware, each checked against its source; for the path protocol we compute exact detection probabilities and bytes on our graphs and take its time per path from Anchuri et al. The basic protocol was measured at commit 9401431; the released code (1,560 tests) adds the optimisations and reproduces the basic proofs byte for byte. All tables and figures are generated from the released raw measurements.

5.2 Soundness in practice

With the basic protocol the verifier accepted all 8,709 honest queries and rejected all 2,895 attacks. On the image models we ran, at $\lambda=40$ with committed weights, one changed value, one changed penultimate neuron, one changed logit, 1% of the weights perturbed, the substitute $\tilde{M}$, and two forgeries that each pass every check but one (2,091 attacks). Every language-model run added +1 to one middle-layer pre-activation (246 attacks, 20 of them on 1- and 2-block builds of the 30–70B shapes). For five models from OPT-1.3B to Llama-2-13B we also ran the image-model attacks that apply to them, for 804 language-model attacks in all. Freivalds’ check rejected 2,747 attacks, and the targeted forgeries were caught where intended: the Reed–Solomon check rejected all 74 forged folded rows and the Merkle check all 74 forged columns, so no part of the protocol is redundant.

Figure 3 plots security against bytes. The basic protocol overshoots its target because r and t are rounded up (48–58 bits at $\lambda=40$, 150–153 at $\lambda=128$), and at $\lambda=40$ it sends $1.1\text{--}8.3\times$ fewer bytes than the paths need for 2^{-40} . With the optimised protocol the achieved security lands closer to the target (43–47 and 131–141 bits), and on every model we ran it on, 2^{-128} costs fewer bytes than the paths need for 2^{-40} .

5.3 Cost of the basic protocol

Image models. With committed weights (Table 2) the prover needs 3.1–27 ms per query and the verifier 5.0–175 ms. The proof, 131 kB–12.1 MB, is $2.4\text{--}4.4\times$ smaller than the int8 model for the VGGs and the CIFAR ResNet, and about its size for the 224-pixel ResNet, whose claims grow with the number of pixels.

Optimised: 128 bits cost fewer bytes than the paths’ 40

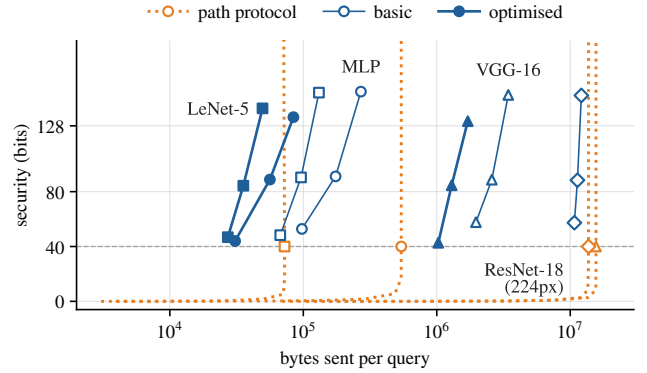


Figure 3: Security against bytes sent per query. The path protocol (dotted) reaches 40 bits only by opening the whole model and trace (vertical lines); the optimised protocol (filled) reaches 128 bits with fewer bytes than that (hollow: basic; $\lambda = 40, 80, 128$).

Table 2: Image models: one query at $\lambda=128$, basic protocol. Proving takes milliseconds, and for the CIFAR models the proof is smaller than the int8 model.

Model	Params	int8	committed (C)			K_{pre}	
			prove	verify	proof	verify	proof
MLP (MNIST)	536K	97.7%	3.1 ms	5.0 ms	270 kB	0.5 ms	3.1 kB
LeNet-5 (MNIST)	62K	99.3%	4.3 ms	7.0 ms	131 kB	1.7 ms	26.1 kB
VGG-11 (CIFAR)	9.8M	91.1%	14.7 ms	49.7 ms	2.2 MB	11.8 ms	610 kB
VGG-16 (CIFAR)	15.2M	93.0%	21.5 ms	63.4 ms	3.4 MB	19.4 ms	1.1 MB
ResNet-18 (CIFAR)	11.2M	94.2%	26.1 ms	110 ms	4.6 MB	46.5 ms	2.5 MB
ResNet-18 (224px) [†]	11.2M	95.0%	27.4 ms	175 ms	12.1 MB	105 ms	9.9 MB

[†]Our ImageNet-trained counterpart of the classifier of [2].

Table 3: Language models: next-token logits of one prompt at $\lambda=128$, basic protocol, random int8 weights (GPU: verifier on the GPU). Proving takes under 30 s, and at 2,048 tokens the claims make up most of the proof.

Model	Params	committed weights (C)				K _{pre}	
		prove	verify	GPU	proof	verify	proof
64-token prompts							
GPT-2	124M	173 ms	468 ms	427 ms	62.1 MB	137 ms	21.8 MB
Qwen3-4B	4.0B	1.9 s	4.6 s	–	659 MB	1.6 s	284 MB
Llama-2-7B	6.7B	2.7 s	4.4 s	3.8 s	762 MB	1.5 s	349 MB
Llama-2-13B	13.0B	4.6 s	6.7 s	–	1.2 GB	2.7 s	547 MB
2,048-token prompts							
OPT-125M	125M	1.1 s	14.2 s	691 ms	733 MB	12.7 s	692 MB
OPT-1.3B	1.3B	5.7 s	78.9 s	3.0 s	3.8 GB	81.2 s	3.7 GB
OPT-6.7B	6.7B	16.2 s	2.7 min	6.4 s	10.1 GB	2.7 min	9.7 GB
Llama-2-7B	6.7B	17.6 s	2.8 min	6.9 s	11.6 GB	2.7 min	11.2 GB
Llama-2-13B	13.0B	27.1 s	3.7 min	–	18.1 GB	3.7 min	17.5 GB

Language models. GPT-2 with a 64-token prompt is proved in 173 ms and verified in 468 ms with a 62.1 MB proof (Table 3), and Llama-2-7B in 2.7 s and 4.4 s with a 762 MB proof, about $9\times$ smaller

Table 4: Basic $\rightarrow$ optimised protocol (C, interactive, $\lambda=128$ unless marked). In C below 2,048 tokens the proof shrinks 2.0–4.2 $\times$ and verification speeds up 2.7–8.4 $\times$.

Model (tokens)	prove	verify	proof
LeNet-5	4.3 $\rightarrow$ 5.3 ms	7.0 $\rightarrow$ 2.4 ms	131 $\rightarrow$ 49.5 kB
VGG-16	21.5 $\rightarrow$ 17.3 ms	63.4 $\rightarrow$ 14.9 ms	3.4 $\rightarrow$ 1.7 MB
ResNet-18 (CIFAR)	26.1 $\rightarrow$ 20.6 ms	110 $\rightarrow$ 24.7 ms	4.6 $\rightarrow$ 2.3 MB
GPT-2 (64)	173 $\rightarrow$ 80.2 ms	468 $\rightarrow$ 92.0 ms	62.1 $\rightarrow$ 14.8 MB
GPT-2 (512)	163 $\rightarrow$ 134 ms	1,012 $\rightarrow$ 375 ms	213 $\rightarrow$ 89.5 MB
Llama-2-7B (1)	2.7 $\rightarrow$ 1.5 s	2,527 $\rightarrow$ 299 ms	418 $\rightarrow$ 130 MB
Llama-2-7B (64)	2.7 $\rightarrow$ 1.7 s	4.4 $\rightarrow$ 1.1 s	762 $\rightarrow$ 329 MB
Llama-2-7B (2,048) ^e	18.7 $\rightarrow$ 5.4 s	6.9 $\rightarrow$ 1.7 s	11.6 GB
Qwen3-4B (8) ^k	135 $\rightarrow$ 119 ms	263 $\rightarrow$ 156 ms	36.1 $\rightarrow$ 20.3 MB

Optimised: planning rule and compact encoding; GPT-2 and Qwen3-4B also compute the last block at the last position only. ^eGPU verifier, streaming when optimised; proof unchanged. ^kK_{pre}, $\lambda=40$, one verifier thread, embedding look-ups.

than its int8 weights. At 2,048 tokens Llama-2-7B is still proved in 17.6 s, but the proof grows to 11.6 GB and CPU verification to 2.8 min, because the verifier recomputes attention over all token pairs; a GPU verifier is 21–41 $\times$ faster there and 1.1–1.3 $\times$ at 64 tokens. Committed weights make the prover of billion-parameter models 10–26 $\times$ slower than known weights at up to 64 tokens, where opening the weights dominates, and 1.0–1.8 $\times$ at 2,048. Prompts whose claims are all sent before the challenge can share one set of challenges and columns: in batches of eight 64-token prompts, Llama-2-7B costs 0.40 s and 401 MB per prompt. For Llama-2-70B the extrapolated proof at 64 tokens is 3.7 GB, 18 $\times$ smaller than its int8 weights.

Figure 4 places these runs among the published ones. Part of the prover gap is our GPU, since all the zkSNARKs except zkLLM ran on CPUs. The orange cross is one path of [2] as reported on an RTX 3090: cheap, but it catches the single-neuron attack on Llama-2-7B with probability 1/11,008.

5.4 The optimised protocol

We re-ran the image models other than the 224-pixel ResNet, GPT-2 at 64 and 512 tokens, Llama-2-7B at 1 and 64 tokens, Qwen3-4B in Maverick’s setting and five of zkLLM’s models at 2,048 tokens with the optimisations, on the same machines. All 4,380 honest queries were accepted and all 4,973 attacks rejected, 3,300 of them on image models under the planning rule. With committed weights, for images and prompts of up to 512 tokens, the proof becomes 2.0–4.2 $\times$ smaller and verification 2.7–8.4 $\times$ faster (Table 4). Much of the verifier’s gain comes from the verifier engineering alone, which also speeds up the basic proofs (468 to 197 ms for GPT-2 at 64 tokens). At 2,048 tokens the claims are about 95% of the proof, which keeps its size, but Llama-2-7B is proved in 5.4 s instead of 18.7 s and verified in 1.7 s instead of 6.9 s. Both gains come from the int32 attention, which the prover shares with the verifier, and from the streaming verifier; we did not separate the two. The committed-weights prover then takes 1.9 $\times$ the plain int8 forward pass. In exchange the one-time setup is longer (114–198 s instead of 10–11 s for GPT-2, about 8.5 instead of 4.5 min for Llama-2-7B), and one cell regressed. For Llama-2-13B in setting C the streaming verifier

Table 5: Optimised protocol against published systems on matched models (theirs/ours; $\uparrow$: ours better; bold: better on all three; ours prove the next-token logits). Our prover is faster everywhere; our proof is smaller only in the bold rows and against Maverick.

System	Model (tokens)	prover	verifier	proof
zkCNN	LeNet-5	81$\times$$\uparrow$	2.2$\times$$\uparrow$	1.1$\times$$\downarrow$
Bionetta	LeNet-5	690 $\times$ $\uparrow$	3.8 $\times$ $\uparrow$	72 $\times$ $\downarrow$
zkCNN	VGG-16	4,481 $\times$ $\uparrow$	3.2 $\times$ $\uparrow$	6.4 $\times$ $\downarrow$
ZKML	VGG-16	32,334 $\times$ $\uparrow$	1.9 $\times$ $\downarrow$	180 $\times$ $\downarrow$
zkGPT	GPT-2 (64)	239 $\times$ $\uparrow$	3.3 $\times$ $\uparrow$	164 $\times$ $\downarrow$
DeepProve	GPT-2 (64)	375$\times$$\uparrow$	13$\times$$\uparrow$	1.3$\times$$\uparrow$
DeepProve	GPT-2 (512)	973 $\times$ $\uparrow$	3.8 $\times$ $\uparrow$	3.6 $\times$ $\downarrow$
ZKTorch ^d	Llama-2-7B (1)	1,786 $\times$ $\uparrow$	334 $\times$ $\uparrow$	5.7 $\times$ $\downarrow$
Maverick ^b	Qwen3-4B (8)	3.0 $\times$ $\uparrow$	1.2 $\times$ $\downarrow$	1.8 $\times$ $\uparrow$
zkLLM	OPT-125M (2,048)	275 $\times$ $\uparrow$	2.1 $\times$ $\uparrow$	5,195 $\times$ $\downarrow$
zkLLM	OPT-1.3B (2,048)	142 $\times$ $\uparrow$	1.2 $\times$ $\uparrow$	25,898 $\times$ $\downarrow$
zkLLM	OPT-6.7B (2,048)	106 $\times$ $\uparrow$	1.4 $\times$ $\uparrow$	64,338 $\times$ $\downarrow$
zkLLM	Llama-2-7B (2,048)	115 $\times$ $\uparrow$	1.4 $\times$ $\uparrow$	63,313 $\times$ $\downarrow$
zkLLM	Llama-2-13B (2,048)	53 $\times$ $\uparrow$	2.3 $\times$ $\downarrow$	96,436 $\times$ $\downarrow$
zkLLM ^c	Llama-2-7B (2,048)	157 $\times$ $\uparrow$	–	–

^aInteractive (Fiat–Shamir not run). ^bK_{pre}, $\lambda=40$, one client thread; Maverick’s time includes its non-linear replay. ^czkLLM’s public code on our L40S.

takes 9.1 s, 5.4 $\times$ Llama-2-7B’s, and host memory peaks at 63 GiB, for reasons we have not found.

5.5 Comparison with published systems

Table 5 compares the optimised protocol with published systems, each in the matching setting: Fiat–Shamir against non-interactive systems, the interactive protocol against zkLLM and Maverick, which are interactive themselves, and committed weights except against Maverick, whose client knows the weights. Our verifier uses 8 CPU threads (one against Maverick, like its client), or the GPU at 2,048 tokens. Prompt lengths match, except that zkGPT states none and we use 64 tokens.

Prover. Ours is faster than every zkSNARK prover on its matched model, by 53 $\times$ (zkLLM, Llama-2-13B) to 32,000 $\times$ (ZKML, VGG-16). Part of this is hardware, so we also ran zkLLM’s public code on our L40S. Its demo proves the components of one layer separately and reports no verifier time, so we time one layer (median of three runs on each of two layers) and multiply by the number of layers. The demo lacks zkLLM’s cross-layer optimisation and the attention output projection, and its attention step writes no output, so we count a layer’s second skip connection twice. For Llama-2-7B at 2,048 tokens this gives about 844 s, 157 $\times$ our 5.4 s (48 $\times$ the basic protocol’s 17.6 s). We do not use the demo for Llama-2-13B, whose time per layer grows out of proportion (2.5 $\times$ the 7B figure).

Verifier and proof. Our verifier is faster than all but three, ZKML’s on VGG-16 (1.9 $\times$), zkLLM’s on Llama-2-13B (2.3 $\times$) and Maverick’s (1.2 $\times$). Maverick’s time, 124.5 ms, is its 87.1 ms of matrix checks plus the 37.4 ms in which its client replays the non-linear operations, work that our verifier also does; ours takes 155.5 ms. On LeNet-5 and on GPT-2 with a 64-token prompt the optimised protocol wins on all three costs: 5.4 ms, 2.6 ms and 63.3 kB against zkCNN’s 441 ms, 5.8 ms and 71.3 kB, and 91 ms, 105 ms and

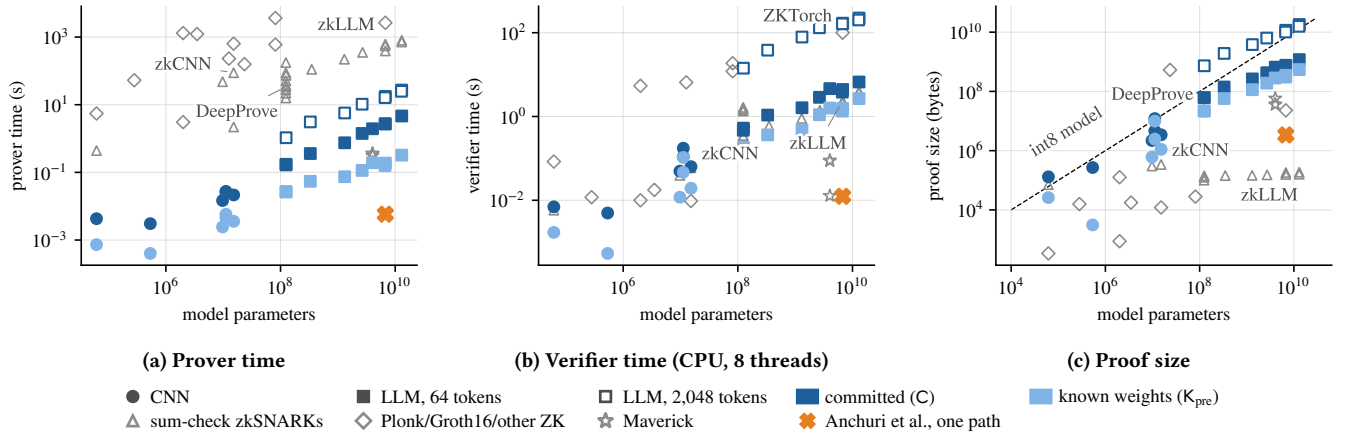


Figure 4: Cost against model size (basic protocol; published systems as reported on their own hardware). Our prover is one to four orders of magnitude below the zkSNARK provers and our verifier is in their range, while our proof is usually larger and grows with the activations.

16.6 MB against DeepProve’s 34.2 s, 1.35 s and 21.7 MB. Both wins come with caveats. zkCNN’s verifier ran on one core and ours on eight threads, and zkCNN’s protocol has a zero-knowledge variant, which ours lacks. DeepProve’s figures are for its transparent (BaseFold) configuration, whose proof certifies every position of its 64-token sequence where ours certifies the next-token logits; its HyperKZG configuration, which needs a structured reference string, sends 9.4 MB. Elsewhere the proof is our cost, 3.6–180× larger than the other short-input proofs and 5,200–96,000× larger than zkLLM’s at 2,048 tokens. In Maverick’s own setting our proof is 1.8× smaller and our prover, on the GPU, 3.0× faster than its 64-thread CPU prover, but Maverick draws a fresh challenge per query, whereas K_{pre} reuses a secret one.

6 Discussion

What this means. Sampling-based proofs of inference are sound only against a weak adversary: they catch a provider who honestly runs the wrong model, but not one who runs the committed model and edits the answer. None of the sampling rules we analysed closes that gap at sub-linear cost. Full soundness does not require a zkSNARK’s prover cost. One random linear combination per weight layer gives a $2^{-\lambda}$ guarantee with no trusted setup and no assumption about how models behave. The client it suits is a gateway or an auditor with a GPU and a fast link rather than a phone, because at long prompts the verifier still recomputes attention. Large models with short prompts suit it best, since there the optimised proof is far smaller than the model (21× for Llama-2-7B at 64 tokens).

Limitations. (i) The proof grows with the activations; at 2,048 tokens it is about 95% claims and gigabytes for 7B models, against zkLLM’s 157–183 kB. (ii) The verifier’s cost grows with the square of the prompt length: at 2,048 tokens the CPU verifier needs minutes, and on a GPU the optimised verifier beats zkLLM’s on four of five models but is 2.3× slower on Llama-2-13B. (iii) The provider must serve the integer model, which changed CNN accuracy by at

most 0.4 points and raised the perplexity of real OPT checkpoints by 7–33% (constants chosen on the evaluation text); the LLM costs were measured with random weights. (iv) Like the path protocol we assume an honest weight commitment, which the optimised codes make up to 19× slower to compute; a malformed one would need a proximity argument as in Ligerio. (v) The opened columns reveal random linear combinations of each weight row, so the protocol is not zero-knowledge. (vi) We prove one forward pass, without a KV cache. (vii) The prototype is Python, and the same-GPU zkLLM comparison rests on its demo.

Future work. The claims, which the compact encoding only halves, could be replaced by a sum-check argument for the non-weight operations; a native verifier and a KV cache would also help, and per-channel normalisation gains may narrow the perplexity gap at no proving cost. Before any wider publication we will share the attack with the authors of [2].

7 Conclusion

A single tampered neuron defeats the lightweight path-sampling proof of inference of [2], on an MNIST network and on the real OPT-6.7B, and no sampling rule we analysed repairs it at sub-linear cost. Checking every layer with Freivalds’ algorithm against a Reed–Solomon/Merkle commitment does: our verifier accepts a wrong output with probability at most $2^{-\lambda}$ and rejected all 2,895 attacks, our prover is one to four orders of magnitude faster than published zkSNARK provers, and the optimised protocol also beats zkCNN on LeNet-5 and DeepProve’s transparent configuration on GPT-2 in verifier time and proof size. Long prompts, where the proof is mostly claims, remain its weak point.

8 Contributions

[To be completed by the team.] Ofek Lavi: [...], Erel Barzilay: [...], Edo Kadosh: [...].

References

- [1] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. 2017. Ligerio: Lightweight Sublinear Arguments Without a Trusted Setup. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2087–2104. doi:10.1145/3133956.3134104
- [2] Pranay Anchuri, Matteo Campanelli, Paul Cesaretti, Rosario Gennaro, Tushar M. Jois, Hasan S. Kayman, and Tugce Ozdemir. 2026. Towards Verifiable AI with Lightweight Cryptographic Proofs of Inference. In *2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 955–977. doi:10.1109/SaTML68715.2026.00058
- [3] Arasu Arun, Adam St. Arnaud, Alexey Titov, Brian Wilcox, Viktor Kolobaric, Marc Brinkmann, Oguzhan Ersoy, Ben Fielding, and Joseph Bonneau. 2025. Verde: Verification via Refereed Delegation for Machine Learning Programs. arXiv:2502.19405.
- [4] Wyatt Benno, Alberto Centelles, Antoine Douchet, and Khalil Gibran. 2026. Jolt Atlas: Verifiable Inference via Lookup Arguments in Zero Knowledge. arXiv:2602.17452.
- [5] Bing-Jyue Chen, Lilia Tang, and Daniel Kang. 2025. ZKTorch: Compiling ML Inference to Zero-Knowledge Proofs via Parallel Proof Accumulation. arXiv:2507.07031.
- [6] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. ACM, 560–574. doi:10.1145/3627703.3650088
- [7] Ziqun Chen, Ming Wu, Michael Heinrich, Jason Zeng, Huiying Lan, Tianwei Zhang, and Rui Tan. 2026. Communication-Efficient Verifiable Attention for LLM Inference. arXiv:2606.16352. Presents VeriAttn.
- [8] KD Conway, Cathie So, Xiaohang Yu, and Kartini Wong. 2024. opML: Optimistic Machine Learning on Blockchain. arXiv:2401.17555.
- [9] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. 2021. ZEN: An Optimizing Compiler for Verifiable, Zero-Knowledge Neural Network Inferences. Cryptology ePrint Archive, Paper 2021/087. <https://eprint.iacr.org/2021/087>
- [10] Boyuan Feng, Zheng Wang, Yuke Wang, Shu Yang, and Yufei Ding. 2024. ZENO: A Type-based Optimization Framework for Zero Knowledge Neural Network Inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 450–464. doi:10.1145/3617232.3624852
- [11] Amos Fiat and Adi Shamir. 1986. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology – CRYPTO ’86 (LNCS, Vol. 263)*. Springer, 186–194.
- [12] Rusins Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. In *Information Processing 77, Proceedings of the IFIP Congress*. North-Holland, 839–842.
- [13] Nicolas Gailly, Ismael Hishon-Rezaizadeh, Tianyi Liu, Nicholas Mainardi, Dimitrios Papadopoulos, Charalampos Papamanthou, Christodoulos Pappas, Shravan Srinivasan, Zack Youell, and Yupeng Zhang. 2026. DeepProve: Verifiable End-to-End Large Language Model Inference. Cryptology ePrint Archive, Paper 2026/1112. To appear at ACM CCS 2026.
- [14] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. 2017. SafetyNets: Verifiable Execution of Deep Neural Networks on an Untrusted Cloud. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [15] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2008. Delegating Computation: Interactive Proofs for Muggles. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 113–122.
- [16] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.
- [17] Kyeongtae Lee, Byeongkyu Han, Jihye Kim, and Hyunok Oh. 2026. LAMP: Linear Verification of Matrix Multiplication via Proximity Testing. Cryptology ePrint Archive, Paper 2026/1571.
- [18] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270. doi:10.1109/TDSC.2023.3348760
- [19] Youki Lim and Sam Yong. 2026. Seal, Then Sample: Sampled Layerwise Proofs for Verifiable LLM Inference from GPT-2 to 70B. arXiv:2609.27367.
- [20] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2968–2985. doi:10.1145/3460120.3485379
- [21] Ben Merbaum, Mohammad Amin Raeisi, Wenhao Wang, Charalampos Papamanthou, Katerina Sotiraki, and Fan Zhang. 2026. Maverick: Private and Verifiable LLM Inference Made Practical via Matrix-Vector Multiplication Delegation. arXiv:2609.10264.
- [22] Ralph C. Merkle. 1987. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology – CRYPTO ’87 (LNCS, Vol. 293)*. Springer, 369–378.
- [23] Jack Min Ong, Matthew Di Ferrante, Aaron Pazdera, Ryan Garner, Sami Jaghouar, Manveer Basra, Max Ryabinin, and Johannes Hagemann. 2025. TOPLOC: A Locality Sensitive Hashing Scheme for Trustless Verifiable Inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. 47196–47211.
- [24] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. 2025. zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference. In *34th USENIX Security Symposium*. USENIX Association, 2045–2063.
- [25] Irving S. Reed and Gustave Solomon. 1960. Polynomial Codes over Certain Finite Fields. *J. Soc. Indust. Appl. Math.* 8, 2 (1960), 300–304.
- [26] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkLLM: Zero Knowledge Proofs for Large Language Models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 4405–4419. doi:10.1145/3658644.3670334
- [27] Yifan Sun, Yuhang Li, Yue Zhang, Yuchen Jin, and Huan Zhang. 2024. SVIP: Towards Verifiable Inference of Open-source Large Language Models. arXiv:2410.22307.
- [28] Florian Tramèr and Dan Boneh. 2019. Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware. In *International Conference on Learning Representations (ICLR)*.
- [29] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In *30th USENIX Security Symposium*. USENIX Association, 501–518.
- [30] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. 2025. zkPyTorch: A Hierarchical Optimized Compiler for Zero-Knowledge Machine Learning. Cryptology ePrint Archive, Paper 2025/535.
- [31] Jiaqi Xue, Yifei Zhao, Mengxin Zheng, Fan Yao, Yan Solihin, and Qian Lou. 2025. Securing Transformer-based AI Execution via Unified TEEs and Crypto-protected Accelerators. arXiv:2507.03278. Presents TwinShield.
- [32] Jianzhu Yao, Hongxu Su, Taobo Liao, Zerui Cheng, Huan Zhang, Xuechao Wang, and Pramod Viswanath. 2026. TAO: Tolerance-Aware Optimistic Verification for Floating-Point Neural Networks. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. ACM, 1515–1532. doi:10.1145/3767295.3803612
- [33] Dmytro Zakharov, Oleksandr Kurbatov, Artem Sdobnov, Lev Soukhanov, Yevhenii Sekhin, Vitalii Volovyk, Mykhailo Velykodnyi, Mark Cherepovskyi, Kyrlo Baibula, Lasha Antadze, Pavlo Kravchenko, Volodymyr Dubinin, and Yaroslav Panasenko. 2025. Bionetta: Efficient Client-Side Zero-Knowledge Machine Learning Proving. arXiv:2510.06784.
- [34] Zkonduit. 2026. EZKL: An Engine for Doing Inference for Deep Learning Models and Other Computational Graphs in a zk-SNARK. <https://github.com/zkonduit/ezkl>. GitHub repository, accessed September 2026.